

FIDUCIA

Data Quality Management System

Technical Specification

Logging, Audit Trail & Notification Services

Document Title	FDQ — Technical Specification: Activity Logging, Error Logging, Event Sourcing & Notification Services
Version	1.0 — Initial Release
Date	May 2026
Prepared For	Development Team — Fiducia DQMS
Source FRD	Fiducia DQMS FRD v1.1 (March 2026)
FR References	FR-SEC-05, FR-STORE-02/03, FR-VER-10, FR-NOTIF-01/02/04/05/06, FR-ETL-06, FR-REM-14/17
Classification	CONFIDENTIAL — Internal Engineering Use Only

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Purpose of This Document	4
1.2 System Context	4
1.3 Technology Stack	4
2. Authentication, Authorisation & Security Foundation	6
2.1 Authentication	6
2.2 Authorisation Scopes	6
2.3 Standard Error Response Envelope	6
2.4 Rate Limiting	6
3. Activity Logging Service	8
3.1 Purpose & Design Rationale	8
3.2 Database Schema — activity_logs	8
3.3 API Endpoints	9
3.3.1 POST /api/v1/activity-logs — Record an Activity Log Entry	9
3.3.2 GET /api/v1/activity-logs — Query Activity Logs	9
3.3.3 GET /api/v1/activity-logs/{log_id} — Retrieve Single Entry	10
3.3.4 GET /api/v1/activity-logs/summary — Aggregate Statistics	10
4. Error Logging Service	11
4.1 Purpose & Design Rationale	11
4.2 Database Schema — error_logs	11
4.3 API Endpoints	11
4.3.1 POST /api/v1/error-logs — Record an Error	11
4.3.2 GET /api/v1/error-logs — Query Error Logs	12
4.3.3 PATCH /api/v1/error-logs/{error_id}/status — Update Resolution Status	12
4.3.4 GET /api/v1/error-logs/stats — Error Statistics	12
5. Event Sourcing & Audit Trail Service	13
5.1 Purpose & Design Rationale	13
5.2 Database Schema — audit_events	13
5.3 API Endpoints	14
5.3.1 POST /api/v1/audit-events — Append an Audit Event	14
5.3.2 GET /api/v1/audit-events — Query Audit Events	15
5.3.3 GET /api/v1/audit-events/{event_id} — Retrieve Single Event	15
5.3.4 GET /api/v1/audit-events/entity/{aggregate_type}/{aggregate_id} — Full Entity History	15
5.3.5 POST /api/v1/audit-events/verify — Chain Integrity Verification	15
6. Notification Service	16
6.1 Purpose & Design Rationale	16

6.2 Database Schema — notification_logs	16
6.3 API Endpoints	16
6.3.1 POST /api/v1/notifications/dispatch — Universal Notification Dispatch	16
6.3.2 POST /api/v1/notifications/email — Direct Email Send (System Admin only)	17
6.3.3 POST /api/v1/notifications/teams — Direct Teams Message (System Admin only)	17
6.3.4 GET /api/v1/notifications/{notification_id}/status — Delivery Status	17
6.3.5 GET /api/v1/notifications/history — Notification History	18
6.3.6 PUT /api/v1/notifications/preferences/{user_id} — User Preferences	18
6.3.7 GET/PUT /api/v1/notifications/templates/{template_id} — Template Management	18
7. Implementation Priorities & Dependency Order	19
8. Code Examples	20
8.1 Activity Logging POST Endpoint (FastAPI)	20
8.2 Audit Event Append with Hash Chaining (FastAPI)	20
9. Testing Strategy	22
9.1 Unit Tests	22
9.2 Integration Tests	22
9.3 Performance Tests	22
10. Deployment & Configuration	23
10.1 Environment Variables	23
10.2 Health Check Endpoints	23
10.3 Kubernetes Deployment Notes	23
10.4 OpenAPI / Swagger Documentation	23
11. Security Considerations	25
11.1 PII in Log Payloads	25
11.2 Audit Trail Tampering Detection	25
11.3 Log Injection	25
11.4 Data Retention & Purge	25
Appendix A — FDQ Event Type Registry	26
Appendix B — HTTP Status Code Reference	26

1. Introduction

1.1 Purpose of This Document

This Technical Specification defines the complete design, data models, API contract, and implementation guidance for four interconnected services within the Fiducia Data Quality Management System (FDQ): the Activity Logging Service, the Error Logging Service, the Event Sourcing and Audit Trail Service, and the Notification Service.

It is written as an executable handover document — you should be able to open your IDE and begin coding immediately after reading it. Every endpoint is defined with HTTP method, path, authentication scope, request schema, response schema, status codes, and the business reason behind each design decision.

1.2 System Context

The FDQ platform is an on-premise, microservices-based Data Trust Platform for financial institutions, subject to CBN KYC regulations, NDPR, AML data integrity requirements, and ISO 27001. The four services defined here are cross-cutting infrastructure that every other FDQ microservice depends on:

- Activity Logging — captures every meaningful user or system action for operational visibility and compliance (maps to FR-SEC-05, FR-VER-10).
- Error Logging — captures exceptions, failures, and warnings with full context so on-call engineers can diagnose issues without needing to reproduce them (maps to FR-ETL-06, FR-ORCH-04).
- Event Sourcing / Audit Trail — maintains an immutable, append-only, hash-chained record of every state change in the system. This is the tamper-evident audit log required by NDPR (7-year retention), CBN, and ISO 27001 (maps to FR-SEC-05, FR-STORE-02, FR-STORE-03).
- Notification Service — delivers email and MS Teams alerts for scan completions, critical data quality issues, SLA breaches, ETL failures, and customer-facing remediation updates (maps to FR-NOTIF-01, FR-NOTIF-02, FR-NOTIF-04).

Design Principle

These four services are write-heavy and read-occasionally. Optimise for write throughput and durability first. Never sacrifice data integrity for performance.

1.3 Technology Stack

Layer	Technology	Rationale
API Framework	FastAPI (Python 3.11+)	Async-first, native OpenAPI generation, Pydantic v2 validation, minimal overhead.
Database	PostgreSQL 15+	Proven ACID guarantees, JSONB for flexible metadata, native INET type for IP storage, row-level security.
Async Queue	Celery + Redis	Notification delivery and async log ingestion decoupled from request path to meet 2s p95 SLA.
Auth	OAuth 2.0 / JWT (RS256)	Stateless tokens; scopes enforce least-privilege access per service.
Notification Transport	SMTP / MS Teams Webhooks	On-premise SMTP relay (no SaaS dependency). Teams Incoming Webhooks need no agent installation.

Log Aggregation	ELK Stack	Structured JSON logs from all services centralised for operational monitoring (NFR 20.5).
-----------------	-----------	---

2. Authentication, Authorisation & Security Foundation

All four services share a common security model. You must implement this before building any endpoint.

2.1 Authentication

Every request to the four services must carry a valid JWT Bearer token issued by the institution's Identity Provider (IdP) via OAuth 2.0. The API Gateway (Kong or NGINX) validates the token signature (RS256) and rejects invalid tokens with HTTP 401 before the request reaches any service.

Internal services communicating with each other use the OAuth 2.0 Client Credentials flow. Tokens have a maximum TTL of 3,600 seconds. Do not cache tokens beyond their expiry.

2.2 Authorisation Scopes

Scope	Grants Access To	Roles Assigned
logs:write	POST to activity-logs and error-logs	All internal services (via client credentials)
logs:read	GET from activity-logs and error-logs	Data Analyst, Compliance Officer, System Admin, DGS
audit:append	POST to audit-events (append only)	All internal services (via client credentials)
audit:read	GET from audit-events	DGS, Compliance Officer, System Admin
audit:verify	POST to audit-events/verify	System Admin, Compliance Officer only
notifications:send	POST to notification dispatch endpoints	All internal services (via client credentials)
notifications:read	GET notification history and status	System Admin, Compliance Officer
notifications:configure	PUT/PATCH templates, preferences, thresholds	System Admin only

2.3 Standard Error Response Envelope

All four services return errors in a consistent JSON envelope. Implement this as a shared Pydantic model in a common library (fdq_commons.models.errors):

```
{
  "error": {
    "code": "VALIDATION_ERROR", // machine-readable code
    "message": "The field 'email' is required.", // human-readable
    "details": [{ "field": "email", "issue": "required" }],
    "trace_id": "a3f8c21d-...", // correlates to audit log
    "timestamp": "2026-03-14T10:22:05.123Z"
  }
}
```

2.4 Rate Limiting

Rate limits are enforced at the API Gateway layer, not within each service. This separates business logic from infrastructure concerns and ensures limits survive service restarts.

Service	Endpoint Group	Default Limit	Burst Limit
---------	----------------	---------------	-------------

Activity Logging	POST (write)	5,000 rpm per client	8,000 rpm
Activity Logging	GET (read)	1,000 rpm per client	2,000 rpm
Error Logging	POST (write)	2,000 rpm per client	4,000 rpm
Audit Events	POST (append)	10,000 rpm per client	15,000 rpm
Audit Events	GET (read)	500 rpm per client	1,000 rpm
Notifications	POST dispatch	1,000 rpm per client	2,000 rpm

Include X-RateLimit-Limit, X-RateLimit-Remaining, and X-RateLimit-Reset headers in every response (FR-REM-17).

3. Activity Logging Service

3.1 Purpose & Design Rationale

The Activity Logging Service answers the question: 'Who did what, when, from where, and to which record?' It captures every meaningful user or system action — record views, approval decisions, pipeline triggers, configuration changes — with enough context to reconstruct a complete operational picture for a compliance auditor or an incident responder.

Unlike the Audit Trail service (Section 5), which is the immutable legal record, the Activity Log is the operational log. It is queryable, filterable, and can be used for dashboards and analytics. It should be written asynchronously via Celery to avoid adding latency to the primary request path.

Implementation Note

Every FDQ microservice (Scan Engine, Remediation Service, Verification Portal, ETL Orchestrator, etc.) must call this service's POST endpoint after completing any user-facing or system-level action. The call must be non-blocking — fire and forget. If the logging call fails, it must not fail the primary operation.

3.2 Database Schema — activity_logs

```
CREATE TABLE activity_logs (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  correlation_id    UUID,                                -- ties a chain of related
events                                                     events
  session_id        UUID,                                -- user session identifier
  service_name       VARCHAR(100) NOT NULL,              -- originating microservice
  event_type         VARCHAR(100) NOT NULL,              -- e.g. RECORD_APPROVED,
SCAN_TRIGGERED                                           SCAN_TRIGGERED
  actor_user_id      UUID,                                -- NULL for system-initiated
events                                                     events
  actor_role         VARCHAR(80),
  actor_ip_address   INET NOT NULL,
  actor_device_info  JSONB,                              -- user-agent, device type, OS
  target_entity_type VARCHAR(100),                       -- e.g. CUSTOMER_RECORD,
SCAN_JOB                                                  SCAN_JOB
  target_entity_id   VARCHAR(255),                       -- primary key of affected
entity                                                     entity
  action             VARCHAR(150) NOT NULL,              -- human-readable action label
  status             VARCHAR(20) NOT NULL                -- SUCCESS | FAILURE | PARTIAL
CHECK (status IN ('SUCCESS', 'FAILURE', 'PARTIAL')),
  failure_reason      TEXT,                              -- populated when status =
FAILURE                                                    FAILURE
  metadata           JSONB,                              -- any extra context as key-
value pairs                                                value pairs
  environment        VARCHAR(20) NOT NULL DEFAULT 'production',
  created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Indexes for common query patterns
CREATE INDEX idx_al_actor_user  ON activity_logs(actor_user_id, created_at DESC);
CREATE INDEX idx_al_entity     ON activity_logs(target_entity_type, target_entity_id);
CREATE INDEX idx_al_event_type ON activity_logs(event_type, created_at DESC);
CREATE INDEX idx_al_correlation ON activity_logs(correlation_id);
CREATE INDEX idx_al_created_at  ON activity_logs(created_at DESC);

-- Partitioning by month is recommended at > 100M rows/year
```


3.3 API Endpoints

3.3.1 POST /api/v1/activity-logs — Record an Activity Log Entry

Auth Scope: logs:write

Idempotency: Clients may include an Idempotency-Key header (UUID). If the same key arrives within 60 seconds, return the cached 201 response without re-inserting.

Request Body

```
{
  "correlation_id": "uuid | null",
  "session_id": "uuid | null",
  "service_name": "string, required, max 100 chars",
  "event_type": "string, required – use FDQ event type registry (see Appendix A)",
  "actor_user_id": "uuid | null",
  "actor_role": "string | null",
  "actor_ip_address": "string, required – IPv4 or IPv6, validated server-side",
  "actor_device_info": { "user_agent": "str", "device_type": "str", "os": "str" },
  "target_entity_type": "string | null",
  "target_entity_id": "string | null",
  "action": "string, required, max 150 chars",
  "status": "SUCCESS | FAILURE | PARTIAL",
  "failure_reason": "string | null",
  "metadata": { "key": "value" } // any JSON object
}
```

Response — 201 Created

```
{ "log_id": "uuid", "created_at": "ISO-8601" }
```

Status Codes

Code	Meaning
201	Entry recorded successfully.
400	Malformed JSON or missing required fields. Returns field-level error details.
401	Missing or invalid Bearer token.
403	Token valid but insufficient scope (logs:write required).
422	Payload passes JSON parse but fails validation (e.g., invalid IP format, unknown status value).
429	Rate limit exceeded. Retry after X-RateLimit-Reset timestamp.
500	Internal error. The logging service must itself emit a structured error to stderr/ELK — never drop silently.

3.3.2 GET /api/v1/activity-logs — Query Activity Logs

Auth Scope: logs:read

Query Params: actor_user_id (uuid), event_type (string), target_entity_id (string), status (string), start_date (ISO-8601), end_date (ISO-8601), service_name (string), page (int, default 1), page_size (int, default 50, max 200)

Response — 200 OK

```
{ "data": [ { /* activity_log object */ } ],
  "pagination": { "page": 1, "page_size": 50, "total": 12450, "total_pages": 249 } }
```

Important: Never return more than 200 records per page regardless of the requested page_size. Use cursor-based pagination (after_id param) for exports exceeding 10,000 records to avoid deep offset performance degradation.

3.3.3 GET /api/v1/activity-logs/{log_id} — Retrieve Single Entry

Auth Scope: logs:read

Path Param: log_id — UUID of the log entry

Response: 200 OK — full activity log object, or 404 Not Found

3.3.4 GET /api/v1/activity-logs/summary — Aggregate Statistics

Auth Scope: logs:read

Query Params: start_date, end_date, group_by (event_type | actor_role | service_name | status)

Response: 200 OK — array of { group_key, count, failure_count, last_occurrence }

Use this endpoint to power the admin console activity heatmap and the Compliance Officer's user activity report. Back it with a materialised view refreshed every 15 minutes to meet the NFR 20.1 response time requirement.

4. Error Logging Service

4.1 Purpose & Design Rationale

The Error Logging Service captures exceptions, failures, and degraded states from all FDQ microservices. Its primary consumers are the on-call engineer (for incident response) and the L2/L3 support tiers. Every error log entry must capture enough information that an engineer can diagnose the root cause without needing to reproduce the issue — this means full stack traces, the full request context (without PII), and the state of the system at the time of failure.

Errors are persisted in a dedicated database table (not the application logs). Application logs (stdout/ELK) are for operational monitoring. Error log entries are the authoritative record of system failures and feed into the SLA monitoring and incident escalation workflows described in NFR 20.7.

4.2 Database Schema — error_logs

```
CREATE TABLE error_logs (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  correlation_id    UUID,
  service_name      VARCHAR(100) NOT NULL,
  error_code        VARCHAR(100) NOT NULL,          -- e.g. ETL_PROPAGATION_FAILED
  error_message     TEXT NOT NULL,
  stack_trace       TEXT,
  severity          VARCHAR(20) NOT NULL
  CHECK (severity IN
('DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL')),
  request_context   JSONB,          -- { endpoint, method, request_id, ip,
  user_agent        }
  actor_user_id     UUID,
  resolution_status VARCHAR(20) NOT NULL DEFAULT 'OPEN'
  CHECK (resolution_status IN
('OPEN', 'ACKNOWLEDGED', 'RESOLVED', 'SUPPRESSED')),
  resolved_by       UUID,
  resolved_at       TIMESTAMPTZ,
  resolution_notes  TEXT,
  recurrence_count  INTEGER NOT NULL DEFAULT 1, -- deduplicated repeats
  first_occurrence  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_occurrence   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  metadata          JSONB,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_el_service_sev ON error_logs(service_name, severity, created_at
DESC);
CREATE INDEX idx_el_status      ON error_logs(resolution_status, severity);
CREATE INDEX idx_el_correlation ON error_logs(correlation_id);
```

Deduplication Strategy

Before inserting a new error, check whether an identical `error_code` + `service_name` combination has been logged in the last 5 minutes. If so, increment `recurrence_count` and update `last_occurrence` rather than creating a new row. This prevents alert storms during cascading failures. Hash the (`error_code` + `service_name` + `stack_trace_first_line`) to build the deduplication key.

4.3 API Endpoints

4.3.1 POST /api/v1/error-logs — Record an Error

Auth Scope: logs:write

Request Body

```
{
  "correlation_id": "uuid | null",
  "service_name": "string, required",
  "error_code": "string, required — use FDQ error code registry",
  "error_message": "string, required",
  "stack_trace": "string | null — full traceback, no truncation",
  "severity": "DEBUG | INFO | WARNING | ERROR | CRITICAL",
  "request_context": {
    "endpoint": "/api/v1/etl/run",
    "method": "POST",
    "request_id": "uuid",
    "ip_address": "string, // of caller or service",
    "user_agent": "string"
  },
  "actor_user_id": "uuid | null",
  "metadata": { "batch_id": "...", "affected_records": 42 }
}
```

Response — 201 Created

```
{ "error_log_id": "uuid", "deduplicated": false, "recurrence_count": 1 }
```

When *deduplicated* is true, the response returns the existing *log_id* and the incremented *recurrence_count*. The service must NOT trigger duplicate alert notifications in this case.

4.3.2 GET /api/v1/error-logs — Query Error Logs

Auth Scope: logs:read

Query Params: service_name, severity (multi-value), resolution_status, start_date, end_date, page, page_size (max 100)

Response — 200 OK

```
{ "data": [ { /* error_log object */ } ],
  "pagination": { "page": 1, "page_size": 50, "total": 88 } }
```

4.3.3 PATCH /api/v1/error-logs/{error_id}/status — Update Resolution Status

Auth Scope: logs:read (DGS, Compliance Officer, System Admin)

This is the only mutation endpoint in the Error Logging Service. It exists so on-call engineers can acknowledge, resolve, or suppress an error. The update is non-destructive — the original error data is preserved.

Request Body

```
{ "resolution_status": "ACKNOWLEDGED | RESOLVED | SUPPRESSED",
  "resolution_notes": "string | null" }
```

Response — 200 OK

```
{ "error_log_id": "uuid", "resolution_status": "RESOLVED", "resolved_at": "ISO-8601" }
```

4.3.4 GET /api/v1/error-logs/stats — Error Statistics

Auth Scope: logs:read

Query Params: start_date, end_date, group_by (service_name | severity | error_code)

Response: 200 OK — { group_key, total_errors, open_count, critical_count, resolved_count, avg_recurrence }

5. Event Sourcing & Audit Trail Service

5.1 Purpose & Design Rationale

This is the most critical service in this specification. The Audit Trail is the tamper-evident, legally admissible record of every state-changing event in the FDQ system. It satisfies FR-SEC-05 (immutable audit log, 7-year retention), FR-STORE-02 (original values never overwritten), FR-STORE-03 (full data lineage and provenance), and the NDPR right-to-audit requirements.

Three design decisions make this different from ordinary logging:

1. Append-only — there are no UPDATE or DELETE endpoints. Once an event is written, it cannot be changed. The database must enforce this at the trigger level, not just the API level.
2. Hash chaining — each event stores the SHA-256 hash of the previous event in the chain for the same aggregate, creating a verifiable chain of custody. Any tampering (even at the database level) breaks the chain and is detected by the /verify endpoint.
3. Before/after state capture — the payload must record the state of the entity before and after the change. This is how a DGS or regulator reconstructs the full history of a customer record.

Critical Constraint

No endpoint in this service may support HTTP PUT, PATCH, or DELETE on event records. If a developer requests such an endpoint, refuse and escalate to the Tech Lead. Audit events are write-once records. This is a regulatory requirement, not a preference.

5.2 Database Schema — audit_events

```
CREATE TABLE audit_events (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  sequence_number    BIGSERIAL          NOT NULL UNIQUE,    -- global ordering, never gaps
  idempotency_key    UUID                NOT NULL UNIQUE,    -- caller-provided, prevents
  duplicates
  event_type         VARCHAR(150)       NOT NULL,           -- e.g. RECORD_FIELD_CORRECTED
  aggregate_type     VARCHAR(100)       NOT NULL,           -- e.g. CUSTOMER_RECORD,
  SCAN_JOB
  aggregate_id       VARCHAR(255)       NOT NULL,           -- PK of the entity being
  described
  actor_user_id      UUID,               -- null for system events
  actor_role         VARCHAR(80),
  actor_ip_address   INET               NOT NULL,
  actor_device_info  JSONB,
  payload            JSONB              NOT NULL,           -- { before: {}, after: {},
  diff: {} }
  schema_version     INTEGER            NOT NULL DEFAULT 1,
  previous_event_hash VARCHAR(64),       -- SHA-256 of prior event for
  aggregate
  event_hash         VARCHAR(64)       NOT NULL,           -- SHA-256(this event canonical
  JSON)
  occurred_at        TIMESTAMPTZ       NOT NULL,           -- business timestamp (when it
  happened)
  recorded_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW(), -- server timestamp (when
  persisted)
  metadata           JSONB
);

-- Prevent any UPDATE or DELETE at the database layer
CREATE OR REPLACE FUNCTION prevent_audit_mutation()
RETURNS TRIGGER AS $$
```

```

BEGIN
    RAISE EXCEPTION 'Audit events are immutable. Mutation denied.';
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_no_update BEFORE UPDATE ON audit_events
    FOR EACH ROW EXECUTE FUNCTION prevent_audit_mutation();
CREATE TRIGGER trg_audit_no_delete BEFORE DELETE ON audit_events
    FOR EACH ROW EXECUTE FUNCTION prevent_audit_mutation();

CREATE INDEX idx_ae_aggregate    ON audit_events(aggregate_type, aggregate_id,
sequence_number);
CREATE INDEX idx_ae_actor        ON audit_events(actor_user_id, occurred_at DESC);
CREATE INDEX idx_ae_event_type  ON audit_events(event_type, occurred_at DESC);

```

5.3 API Endpoints

5.3.1 POST /api/v1/audit-events — Append an Audit Event

Auth Scope: audit:append

Idempotency: Required — caller must provide X-Idempotency-Key header. Duplicate keys within 24 hours return the original 201 response without re-inserting.

Request Body

```

{
  "idempotency_key": "uuid, required",
  "event_type": "string, required — use FDQ event type registry",
  "aggregate_type": "string, required — CUSTOMER_RECORD | SCAN_JOB | RULE | USER | ETL_JOB",
  "aggregate_id": "string, required",
  "actor_user_id": "uuid | null",
  "actor_role": "string | null",
  "actor_ip_address": "string, required",
  "actor_device_info": { "user_agent": "str", "device_type": "str" },
  "payload": {
    "before": { /* entity state before change */ },
    "after": { /* entity state after change */ },
    "diff": { /* field-level diff, optional but recommended */ }
  },
  "schema_version": 1,
  "occurred_at": "ISO-8601, required — the business timestamp",
  "metadata": { }
}

```

Server-Side Hash Computation (you must implement this)

The service — not the caller — computes the hash. Never trust a caller-provided hash. Before inserting, the service must:

- Query the latest audit event for this aggregate_type + aggregate_id to retrieve its event_hash (this becomes previous_event_hash).
- Construct the canonical JSON string: `JSON.dumps(payload_fields, sort_keys=True, ensure_ascii=True)` — deterministic serialisation is critical.
- Compute `event_hash = SHA-256(sequence_number || idempotency_key || aggregate_id || occurred_at || canonical_payload)`.
- Insert the row atomically with the computed hashes inside a database transaction.

Response — 201 Created

```
{ "event_id": "uuid", "sequence_number": 10042, "event_hash": "a3f8c2...",  
  "recorded_at": "ISO-8601" }
```

5.3.2 GET /api/v1/audit-events — Query Audit Events

Auth Scope: audit:read

Query Params: aggregate_type, aggregate_id, event_type, actor_user_id, start_date, end_date, page, page_size (max 100)

Response: 200 OK — paginated list of audit event objects

5.3.3 GET /api/v1/audit-events/{event_id} — Retrieve Single Event

Auth Scope: audit:read

Response: 200 OK — full audit event object including hashes. Never expose raw stack traces or internal system data in payload fields visible to external callers — apply field-level masking for non-admin roles.

5.3.4 GET /api/v1/audit-events/entity/{aggregate_type}/{aggregate_id} — Full Entity History

Auth Scope: audit:read

Returns the complete ordered history of all events for a specific entity (e.g., all events ever recorded for customer record ID CR-00421). This is the primary endpoint used by the Data Lineage Viewer (FR-RPT-07) and DGS case review (FR-VER-02).

Query Params: from_sequence (int), to_sequence (int), event_type (filter), page, page_size

Response: 200 OK — chronologically ordered list of events with sequence_numbers, hashes, and full payloads (masking PII per caller role)

5.3.5 POST /api/v1/audit-events/verify — Chain Integrity Verification

Auth Scope: audit:verify

This endpoint re-computes and validates the hash chain for a specified range of audit events. It is used by the Compliance Officer for regulatory proof-of-integrity and by the System Admin for post-incident forensics. Run it no more than once per hour per aggregate — it is computationally expensive.

Request Body

```
{ "aggregate_type": "string", "aggregate_id": "string",  
  "from_sequence": 1, "to_sequence": 10042 }
```

Response — 200 OK

```
{ "valid": true, "events_verified": 10042, "first_sequence": 1, "last_sequence": 10042,  
  "broken_at_sequence": null, // populated when valid = false  
  "verification_timestamp": "ISO-8601" }
```

6. Notification Service

6.1 Purpose & Design Rationale

The Notification Service is the outbound communication hub for all FDQ alerts and status updates. It supports three delivery channels: email (SMTP relay), Microsoft Teams (Incoming Webhooks), and in-app dashboard alerts. It is responsible for delivery confirmation, retry logic, deduplication, and digest aggregation (FR-NOTIF-01 through FR-NOTIF-06).

Notification delivery is always asynchronous. The POST dispatch endpoint enqueues the notification via Celery and returns immediately with a notification_id. The caller uses the status endpoint to poll for delivery confirmation. This design ensures that a slow SMTP server or a Teams rate limit does not block the primary workflow.

Alert Fatigue Prevention

The notification service must enforce suppression windows and digest mode (FR-NOTIF-06). A critical scan failure that affects 10,000 records must generate exactly one notification, not 10,000. Implement the suppression logic in the Celery task, not in the caller service.

6.2 Database Schema — notification_logs

```
CREATE TABLE notification_logs (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  notification_id    UUID NOT NULL UNIQUE,           -- caller-provided idempotency
  key               channel                VARCHAR(20) NOT NULL
                  CHECK (channel IN ('EMAIL', 'TEAMS', 'SMS', 'DASHBOARD')),
  recipient         VARCHAR(500) NOT NULL,          -- email addr, teams webhook,
  phone            template_id             VARCHAR(100),
  subject           VARCHAR(500),
  body_preview      TEXT,                        -- first 500 chars for audit
  status            VARCHAR(20) NOT NULL DEFAULT 'QUEUED'
                  CHECK (status IN
    ('QUEUED', 'SENT', 'DELIVERED', 'FAILED', 'SUPPRESSED')),
  provider_message_id VARCHAR(255),              -- SMTP Message-ID or Teams
  activity_id       delivery_attempts      INTEGER NOT NULL DEFAULT 0,
  last_attempt_at   TIMESTAMPTZ,
  delivered_at      TIMESTAMPTZ,
  error_message     TEXT,
  triggered_by_event_id UUID REFERENCES audit_events(id),
  triggered_by_user_id UUID,
  metadata          JSONB,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_nl_status      ON notification_logs(status, created_at DESC);
CREATE INDEX idx_nl_channel    ON notification_logs(channel, created_at DESC);
CREATE INDEX idx_nl_event      ON notification_logs(triggered_by_event_id);
```

6.3 API Endpoints

6.3.1 POST /api/v1/notifications/dispatch — Universal Notification Dispatch

Auth Scope: notifications:send

Idempotency: Caller must provide X-Idempotency-Key header as notification_id. Duplicate keys within 24 hours return the cached 202 without re-queuing.

Request Body

```
{
  "notification_id": "uuid, required — use as idempotency key",
  "channel": "EMAIL | TEAMS | DASHBOARD | SMS",
  "recipient": "email address | teams webhook URL | user_id for dashboard",
  "template_id": "string — references notification_templates table",
  "template_data": { "scan_id": "...", "issue_count": 42, "severity": "CRITICAL" },
  "subject": "string | null — for EMAIL channel only",
  "priority": "HIGH | NORMAL | LOW — default NORMAL",
  "triggered_by_event_id": "uuid | null — links to audit_events for traceability",
  "triggered_by_user_id": "uuid | null",
  "suppress_within_seconds": 300, // do not resend same template to same recipient
  within N seconds
  "metadata": { }
}
```

Response — 202 Accepted (async)

```
{ "notification_id": "uuid", "status": "QUEUED", "queued_at": "ISO-8601" }
```

6.3.2 POST /api/v1/notifications/email — Direct Email Send (System Admin only)

Auth Scope: notifications:configure

Used for administrative one-off emails (e.g., test alerts, user provisioning). Normal operational notifications should go through /dispatch with a template_id. This endpoint bypasses the template engine but still logs to notification_logs.

Request Body

```
{ "notification_id": "uuid", "to": ["email@bank.ng"],
  "cc": [], "subject": "string", "html_body": "string", "text_body": "string" }
```

Response — 202 Accepted

```
{ "notification_id": "uuid", "status": "QUEUED" }
```

6.3.3 POST /api/v1/notifications/teams — Direct Teams Message (System Admin only)

Auth Scope: notifications:configure

Posts a structured Adaptive Card to a configured Teams channel via Incoming Webhook. Webhook URLs are stored in configuration, never in the request body — callers reference a channel_key (e.g., 'dq-alerts', 'etl-ops') and the service resolves the URL internally.

Request Body

```
{ "notification_id": "uuid", "channel_key": "dq-alerts",
  "title": "string", "summary": "string", "facts": [{"name": "str", "value": "str"}],
  "severity": "CRITICAL | WARNING | INFO" }
```

6.3.4 GET /api/v1/notifications/{notification_id}/status — Delivery Status

Auth Scope: notifications:read

Response — 200 OK

```
{ "notification_id": "uuid", "channel": "EMAIL", "status": "DELIVERED",
  "delivery_attempts": 1, "delivered_at": "ISO-8601",
  "provider_message_id": "<smtp_message_id>" }
```

6.3.5 GET /api/v1/notifications/history — Notification History

Auth Scope: notifications:read

Query Params: channel, status, recipient, template_id, start_date, end_date, page, page_size

Response: 200 OK — paginated list of notification_log objects

6.3.6 PUT /api/v1/notifications/preferences/{user_id} — User Preferences

Auth Scope: notifications:configure (admin) OR self-service with own user_id

Request Body

```
{ "email_enabled": true, "teams_enabled": false,
  "digest_mode": true, "digest_interval_minutes": 60,
  "suppression_windows": [{ "day": "SATURDAY", "start": "18:00", "end": "23:59" }],
  "alert_types": ["CRITICAL_ISSUE", "SLA_BREACH", "ETL_FAILURE"] }
```

6.3.7 GET/PUT /api/v1/notifications/templates/{template_id} — Template Management

Auth Scope: notifications:configure

Templates are Jinja2 strings stored in the database. GET retrieves the current template. PUT replaces it. All template changes are version-controlled and audit-logged via the Event Sourcing service (aggregate_type: NOTIFICATION_TEMPLATE). A dry-run parameter on PUT renders the template with sample data and returns the output without saving, for preview and testing.

7. Implementation Priorities & Dependency Order

Build in this exact sequence. Later services depend on earlier ones being operational.

Phase	Service	Sprint (est.)	Notes & Dependencies
1	Shared commons library (fdq_commons)	Sprint 1, Week 1-2	Auth models, error envelope, Pydantic base schemas, IP validation utilities, JWT middleware. All other services import this.
1	Database migrations baseline	Sprint 1, Week 1-2	Alembic migrations for all four schemas. Create the prevent_audit_mutation triggers. Seed reference data (event_type registry, notification templates).
2	Activity Logging Service	Sprint 1, Week 3-4	Depends on Phase 1 only. Build this first because every subsequent service will call it. Celery worker must be running.
2	Error Logging Service	Sprint 2, Week 1-2	Can be built in parallel with Activity Logging. Implement deduplication logic before enabling alerts.
3	Event Sourcing / Audit Trail	Sprint 2, Week 3-4	Depends on Phase 1 & 2. Hash computation must be unit-tested exhaustively before going live — a bug in the hash function means the chain is broken from day one.
4	Notification Service	Sprint 3, Week 1-3	Depends on Audit Trail (triggered_by_event_id FK). SMTP relay and Teams webhook must be configured in environment before integration testing.
5	Integration hardening	Sprint 3, Week 4	End-to-end tests: scan completion triggers activity log + audit event + Teams notification. Verify hash chain integrity after 1,000 synthetic events.

8. Code Examples

8.1 Activity Logging POST Endpoint (FastAPI)

The following is a production-ready implementation of the POST `/api/v1/activity-logs` endpoint. Study the patterns — idempotency key handling, async Celery dispatch, and the structured logging call at the end are all intentional.

```
# services/activity_logging/routes.py
from fastapi import APIRouter, Depends, Header, HTTPException, status
from typing import Optional
from uuid import UUID, uuid4
import structlog
from fdq_commons.auth import require_scope
from fdq_commons.models.activity import ActivityLogCreate, ActivityLogResponse
from .service import ActivityLoggingService
from .cache import idempotency_cache

router = APIRouter(prefix='/api/v1/activity-logs', tags=['Activity Logging'])
log = structlog.get_logger()

@router.post('/', status_code=status.HTTP_201_CREATED,
             response_model=ActivityLogResponse)
async def record_activity(
    body: ActivityLogCreate,
    idempotency_key: Optional[str] = Header(None, alias='Idempotency-Key'),
    token_claims=Depends(require_scope('logs:write')),
    svc: ActivityLoggingService = Depends()
):
    # 1. Check idempotency cache (Redis, 60s TTL)
    if idempotency_key:
        cached = await idempotency_cache.get(idempotency_key)
        if cached:
            return cached # return exact prior response

    # 2. Persist (via Celery for non-blocking write)
    try:
        result = await svc.create(body, caller_id=token_claims['sub'])
    except Exception as exc:
        # NEVER fail the caller; log internally and return 500
        log.error('activity_log_write_failed', error=str(exc),
                  correlation_id=body.correlation_id)
        raise HTTPException(status_code=500, detail='Log write failed')

    # 3. Cache idempotency result
    if idempotency_key:
        await idempotency_cache.set(idempotency_key, result, ttl=60)

    # 4. Emit structured log (goes to ELK via stdout)
    log.info('activity_logged', log_id=str(result.log_id),
            event_type=body.event_type, actor=body.actor_user_id)
    return result
```

8.2 Audit Event Append with Hash Chaining (FastAPI)

This is the most sensitive endpoint in the system. The hash chain computation is done inside a serialised database transaction with SELECT FOR UPDATE to prevent race conditions from concurrent appends to the same aggregate.

```
# services/audit_trail/service.py
import hashlib, json
```

```

from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, text
from .models import AuditEvent
from fdq_commons.models.audit import AuditEventCreate

class AuditTrailService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def append_event(self, body: AuditEventCreate) -> AuditEvent:
        async with self.db.begin(): # transaction
            # 1. Lock on aggregate to prevent concurrent hash miscalculation
            await self.db.execute(text(
                'SELECT pg_advisory_xact_lock(hashtext(:key))',
                {'key': f'{body.aggregate_type}:{body.aggregate_id}'})
            ))

            # 2. Retrieve previous event hash for this aggregate
            prev = await self.db.execute(
                select(AuditEvent.event_hash, AuditEvent.sequence_number)
                .where(AuditEvent.aggregate_type == body.aggregate_type,
                    AuditEvent.aggregate_id == body.aggregate_id)
                .order_by(AuditEvent.sequence_number.desc())
                .limit(1)
            )
            prev_row = prev.fetchone()
            previous_hash = prev_row.event_hash if prev_row else None

            # 3. Compute this event's hash (deterministic serialisation)
            canonical = json.dumps({
                'idempotency_key': str(body.idempotency_key),
                'aggregate_id': body.aggregate_id,
                'event_type': body.event_type,
                'occurred_at': body.occurred_at.isoformat(),
                'payload': body.payload,
                'previous_hash': previous_hash or '',
            }, sort_keys=True, ensure_ascii=True)
            event_hash = hashlib.sha256(canonical.encode()).hexdigest()

            # 4. Insert — database triggers reject any UPDATE/DELETE
            event = AuditEvent(
                idempotency_key = body.idempotency_key,
                event_type = body.event_type,
                aggregate_type = body.aggregate_id, # corrected
                aggregate_id = body.aggregate_id,
                actor_user_id = body.actor_user_id,
                actor_role = body.actor_role,
                actor_ip_address = body.actor_ip_address,
                actor_device_info = body.actor_device_info,
                payload = body.payload,
                schema_version = body.schema_version,
                previous_event_hash = previous_hash,
                event_hash = event_hash,
                occurred_at = body.occurred_at,
                metadata = body.metadata,
            )
            self.db.add(event)
            await self.db.flush() # get sequence_number
            return event

```

9. Testing Strategy

9.1 Unit Tests

Use pytest with pytest-asyncio and httpx.AsyncClient. Every endpoint must have tests for: happy path (201/200), missing required fields (422), invalid auth (401), insufficient scope (403), rate limit response headers, and idempotency key replay.

Critical unit tests unique to this specification:

- Audit hash chain: verify that sequential appends produce a valid chain; verify that modifying any field in a prior event breaks verification.
- Audit mutation prevention: verify that direct SQL UPDATE/DELETE on audit_events raises an exception (test the trigger directly, not just the API).
- Error log deduplication: verify that identical error_code + service_name within 5 minutes increments recurrence_count rather than inserting a new row.
- Notification idempotency: verify that posting the same notification_id twice within 24 hours does not enqueue a second Celery task.
- Notification suppression: verify that suppress_within_seconds prevents re-delivery of the same template to the same recipient within the window.

9.2 Integration Tests

Use a real PostgreSQL instance (not mocks) for integration tests — especially for the audit trail, where the database triggers are critical. Use Docker Compose to spin up PostgreSQL + Redis for the CI pipeline.

End-to-end scenario to automate: (1) Trigger a scan completion event; (2) Assert activity log entry created; (3) Assert audit event appended with valid hash; (4) Assert notification queued; (5) Mock SMTP relay confirms delivery; (6) Assert notification status changes to DELIVERED; (7) Call /verify endpoint and assert valid: true.

9.3 Performance Tests

Use Locust to verify the NFR 20.1 requirement (API endpoints respond within 2 seconds at p95). Specifically test: 500 concurrent writers to POST /api/v1/audit-events (the hash lock must not cause timeout) and 100 concurrent readers on GET /api/v1/audit-events/entity/{type}/{id} for an aggregate with 100,000 events.

10. Deployment & Configuration

10.1 Environment Variables

Variable	Service	Description
DATABASE_URL	All	PostgreSQL connection string (use read-write primary for writes, read-replica for reports).
REDIS_URL	All	Redis connection for Celery broker and idempotency cache.
JWT_PUBLIC_KEY_PATH	All	Path to RS256 public key PEM file for token verification.
SMTP_HOST / SMTP_PORT	Notification	On-premise SMTP relay address and port (no cloud SMTP).
SMTP_FROM_ADDRESS	Notification	Sender address for all outbound email.
TEAMS_CHANNEL_WEBHOOKS	Notification	JSON map of channel_key to webhook URL, e.g. {"dq-alerts": "https://..."}.
NOTIFICATION_RETRY_MAX	Notification	Max Celery retry attempts for failed notifications (default: 5).
AUDIT_HASH_ALGORITHM	Audit Trail	Hashing algorithm (default: sha256). Do not change after go-live — it breaks existing chains.
ERROR_DEDUP_WINDOW_SECONDS	Error Logging	Window for error deduplication (default: 300).

10.2 Health Check Endpoints

Each service must expose GET /health (liveness) and GET /ready (readiness). The readiness endpoint must verify database connectivity and Celery queue responsiveness. These are required by the Kubernetes readiness probe and the NFR 20.3 availability requirement.

10.3 Kubernetes Deployment Notes

- Each service runs as an independent Deployment with a minimum of 2 replicas for availability.
- The Audit Trail service requires a PodDisruptionBudget with minAvailable: 1 — it must never be completely unavailable.
- Celery workers for the Notification Service are deployed as a separate Deployment and should auto-scale (HPA) based on the Redis queue depth metric.
- All services must mount the JWT public key as a Kubernetes Secret, not a ConfigMap. Rotate the secret without restarting pods by implementing a key rotation watch in the JWT middleware.
- Database connection pooling: use PgBouncer sidecar or pg_pool to manage connection limits (PostgreSQL default max_connections is 100 — this is exhausted quickly with 2+ services each with 10 workers).
- Enable PostgreSQL row-level security (RLS) on audit_events and activity_logs to enforce data domain isolation at the database layer as an additional security control.

10.4 OpenAPI / Swagger Documentation

FastAPI auto-generates an OpenAPI 3.0 schema at /openapi.json. Add the following to each endpoint decorator to ensure the generated docs are useful to consumers:

- summary — one-line description.
- description — include the FRD requirement IDs (e.g., 'Satisfies FR-SEC-05, FR-STORE-02').
- response_model — always use explicit Pydantic models, never dict.
- responses — document all possible status codes including 422 and 429.

Expose the Swagger UI at /docs in non-production environments only. Disable it in production (set SWAGGER_UI_ENABLED=false) as it exposes your API surface to attackers.

11. Security Considerations

11.1 PII in Log Payloads

The `activity_log.metadata` and `audit_event.payload` fields will contain customer data. You must implement field-level masking before these values are returned via GET endpoints. A caller with `logs:read` scope must not see raw BVN, NIN, account numbers, or phone numbers. Masking rules must be applied at the serialisation layer (Pydantic model `__get_validators__`) so they cannot be accidentally bypassed by a developer returning a raw ORM object.

In non-production environments, all PII fields in these tables must be pseudonymised at ingestion time using a reversible tokenisation scheme (FR-SEC-07). The tokenisation key must be stored in the on-premise KMS.

11.2 Audit Trail Tampering Detection

The hash chain is your primary tampering detection mechanism but it only catches tampering if someone runs `/verify`. Schedule the integrity verification as a weekly automated job (via Apache Airflow) covering the previous 7 days of audit events across all aggregate types. If the job finds a broken chain, it must immediately raise a CRITICAL error log entry and send a Teams notification to the security channel.

11.3 Log Injection

Sanitise all free-text fields (`error_message`, `action`, `failure_reason`, `resolution_notes`) to prevent log injection attacks. A malicious actor could submit a crafted string with newlines or JSON fragments designed to pollute the ELK index. Use a strict whitelist regex for structured fields and HTML-escape free-text fields before storing.

11.4 Data Retention & Purge

All four log tables must retain data for a minimum of 7 years (NDPR and CBN requirement). Implement a scheduled Airflow DAG that archives records older than the retention window to a cold storage partition or separate archive table, then removes them from the hot table. The archive must be encrypted and verifiably complete before deletion from the hot table. The `audit_events` table is never purged — it is append-only forever.

Appendix A — FDQ Event Type Registry

All services must use event_type values from this registry. New event types must be added here and registered in the database reference table before use.

Event Type	Source Service	Description
SCAN_JOB_TRIGGERED	Scan Engine	A DQ scan job was manually or automatically triggered.
SCAN_JOB_COMPLETED	Scan Engine	Scan job finished; issues_found count in metadata.
RECORD_FLAGGED	Flag & Quarantine	A customer record was flagged and quarantined.
RECORD_FIELD_CORRECTED	Remediation Service	A field correction was submitted via any channel.
RECORD_APPROVED	Verification Portal	DGS approved a corrected field value.
RECORD_REJECTED	Verification Portal	DGS rejected a submitted correction.
RECORD_VERIFIED	Verified Storage	Record stored with VERIFIED status.
RECORD_PROPAGATED	ETL Orchestrator	Verified record pushed to core banking system.
ETL_JOB_STARTED	ETL Orchestrator	Batch ETL job commenced.
ETL_JOB_COMPLETED	ETL Orchestrator	ETL batch completed; success/failure counts in metadata.
ETL_JOB_FAILED	ETL Orchestrator	ETL job failed; error_code and affected_batch_id in metadata.
RULE_CREATED	Rule Engine	A new validation rule was created.
RULE_MODIFIED	Rule Engine	An existing rule was updated (new version created).
RULE_ACTIVATED	Rule Engine	A rule was activated and will run in subsequent scans.
USER_LOGIN	UAM Service	User authenticated successfully.
USER_LOGIN_FAILED	UAM Service	Authentication failed (wrong credentials).
USER_PROVISIONED	UAM Service	New user account created.
USER_ROLE_CHANGED	UAM Service	User role or permissions updated.
CONFIG_CHANGED	Admin Console	System configuration parameter updated.
NOTIFICATION_SENT	Notification Service	Outbound notification dispatched.
AUDIT_CHAIN_VERIFIED	Audit Trail Service	Integrity verification completed; result in metadata.

Appendix B — HTTP Status Code Reference

Code	Name	When to use in FDQ services
------	------	-----------------------------

200	OK	Successful GET, PATCH response.
201	Created	Successful POST that creates a new resource (log entry, event).
202	Accepted	POST accepted for async processing (notification dispatch). Does not guarantee delivery.
204	No Content	Not used in these services — always return a response body.
400	Bad Request	Malformed JSON, unparseable request body.
401	Unauthorised	Missing, expired, or invalid Bearer token.
403	Forbidden	Valid token but insufficient OAuth scope or RBAC role.
404	Not Found	Resource with given ID does not exist.
409	Conflict	Idempotency key collision (only if within TTL and payload differs — rare).
422	Unprocessable	Payload parses but fails validation (invalid IP, unknown enum value, required field missing).
429	Too Many Requests	Rate limit exceeded. Include Retry-After header.
500	Internal Server Error	Unexpected server-side failure. Must emit error log internally.
503	Service Unavailable	Database or Celery broker unreachable. Returned by /ready health check.